

Лабораторная работа № 2.

Модульное тестирование с использованием Jest

Тема: Jest, expect(), matchers, hooks, mock functions и unit testing Service layer

Цель: Настроить Jest в ESM-проекте Node.js, написать unit tests для бизнес-логики Internet Shop, использовать matchers, beforeEach(), async assertions и jest.fn(), а также изолировать Product Service от Prisma через dependency injection.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте проект express-shop. В проекте используется ES Modules через "type": "module". В этой лабораторной PostgreSQL запускать не требуется: unit tests должны работать без реальной базы данных.

1. Установите Jest

```
PowerShell  
  
npm install --save-dev jest
```

2. Настройте scripts в package.json

```
package.json  
  
{  
  "type": "module",  
  "scripts": {  
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js",  
    "test:watch": "node --experimental-vm-modules node_modules/jest/bin/jest.js --watch"  
  }  
}
```

Jest 30.x для native ESM запускается через Node.js с параметром `--experimental-vm-modules`.

3. Создайте конфигурацию Jest

```
jest.config.js  
  
export default {  
  testEnvironment: "node",  
  transform: {},  
  clearMocks: true  
};
```

4. Создайте структуру unit tests

```
Структура проекта
```

```
express-shop/  
  services/  
    orderMath.js  
    productService.js  
  repositories/  
    productRepository.js  
  tests/  
    unit/  
      orderMath.test.js  
      productService.test.js  
  jest.config.js  
  package.json
```

5. Используйте функцию calculateOrderTotal()

```
services/orderMath.js  
  
export function calculateOrderTotal(items) {  
  if (!Array.isArray(items)) {  
    throw new TypeError("items must be an array");  
  }  
  
  return items.reduce((sum, item) => {  
    if (!Number.isInteger(item.price) || item.price <= 0) {  
      throw new RangeError("price must be a positive integer");  
    }  
  
    if (  
      !Number.isInteger(item.quantity) ||  
      item.quantity < 1 ||  
      item.quantity > 100  
    ) {  
      throw new RangeError("quantity must be 1..100");  
    }  
  
    return sum + item.price * item.quantity;  
  }, 0);  
}
```

6. Создайте первый Jest test

```
tests/unit/orderMath.test.js  
  
import {  
  describe,  
  expect,  
  test  
} from "@jest/globals";  
import { calculateOrderTotal } from "../../services/orderMath.js";  
  
describe("calculateOrderTotal", () => {  
  test("calculates total for several items", () => {  
    const items = [  
      { price: 10000, quantity: 2 },  
      { price: 5000, quantity: 1 }  
    ];  
  
    const total = calculateOrderTotal(items);
```

```
    expect(total).toBe(25000);  
  });  
});
```

7. Запустите Jest

PowerShell

```
npm test
```

Первый test должен завершиться со статусом PASS.

8. Проверьте failing test

1. Временно измените ожидаемое значение 25000 на 25001.
2. Запустите `npm test`.
3. Найдите `actual` и `expected` в сообщении Jest.
4. Верните правильное ожидаемое значение.
5. Повторно запустите test suite.

9. Добавьте дополнительные matchers

tests/unit/orderMath.test.js

```
test("returns zero for empty array", () => {  
  expect(calculateOrderTotal([])).toBe(0);  
});  
  
test("rejects quantity equal to zero", () => {  
  expect(() =>  
    calculateOrderTotal([  
      { price: 10000, quantity: 0 }  
    ])  
  ).toThrow(/quantity/);  
});  
  
test("rejects non-array input", () => {  
  expect(() => calculateOrderTotal(null))  
    .toThrow(TypeError);  
});
```

10. Создайте Product Service через dependency injection

services/productService.js

```
export function createProductService(productRepository) {  
  return {  
    async getById(id) {  
      const product = await productRepository.findById(id);  
  
      if (!product) {  
        throw new Error("Product not found");  
      }  
    }  
  }  
}
```

```

    return product;
  },

  async create(data) {
    if (data.price <= 0) {
      throw new Error("Invalid product price");
    }

    return productRepository.create(data);
  }
};
}

```

Service получает dependency через аргумент. В production туда можно передать Prisma repository, а в unit test — mock object.

11. Создайте unit test для getById()

```

tests/unit/productService.test.js

import {
  beforeEach,
  describe,
  expect,
  jest,
  test
} from "@jest/globals";
import { createProductService } from "../../services/productService.js";

describe("Product Service", () => {
  let repository;
  let service;

  beforeEach(() => {
    repository = {
      findById: jest.fn(),
      create: jest.fn()
    };

    service = createProductService(repository);
  });

  test("returns product by id", async () => {
    const product = {
      id: 15,
      name: "Notebook",
      price: 420000
    };

    repository.findById.mockResolvedValue(product);

    const result = await service.getById(15);

    expect(result).toEqual(product);
  });
});

```

12. Проверьте аргументы mock function

Дополнение к test getById()

```
expect(repository.findById)
  .toHaveBeenCalledTimes(15);

expect(repository.findById)
  .toHaveBeenCalledTimes(1);
```

Теперь test проверяет не только результат Service, но и правильное взаимодействие с repository.

13. Добавьте negative test для отсутствующего Product

tests/unit/productService.test.js

```
test("throws when product does not exist", async () => {
  repository.findById.mockResolvedValue(null);

  await expect(service.getById(999))
    .rejects.toThrow("Product not found");

  expect(repository.findById)
    .toHaveBeenCalledTimes(999);
});
```

14. Добавьте test для create()

tests/unit/productService.test.js

```
test("creates product", async () => {
  const input = {
    name: "Mouse",
    price: 12000,
    categoryId: 1,
    inStock: true
  };

  const created = {
    id: 20,
    ...input
  };

  repository.create.mockResolvedValue(created);

  const result = await service.create(input);

  expect(result).toEqual(created);
  expect(repository.create)
    .toHaveBeenCalledTimes(1);
});
```

15. Проверьте invalid price

tests/unit/productService.test.js

```
test("rejects product with invalid price", async () => {
  const input = {
    name: "Mouse",
    price: 0,
    categoryId: 1
  };

  await expect(service.create(input))
    .rejects.toThrow("Invalid product price");

  expect(repository.create)
    .not.toHaveBeenCalled();
});
```

Важно проверить отсутствие вызова repository: invalid business input должен быть остановлен до слоя данных.

16. Добавьте несколько return values mock

Пример

```
repository.findById
  .mockResolvedValueOnce({ id: 1, name: "Mouse" })
  .mockResolvedValueOnce({ id: 2, name: "Keyboard" });
```

`mockResolvedValueOnce()` позволяет описывать последовательные ответы dependency для разных вызовов.

17. Проверьте beforeEach()

6. В одном test вызовите `repository.findById`.
7. В следующем test проверьте число вызовов.
8. Убедитесь, что mock state не переносится между tests.
9. Объясните роль `beforeEach()` и `clearMocks: true`.

18. Используйте test.each() для boundary values

tests/unit/orderMath.test.js

```
test.each([
  [1, 1000],
  [100, 100000]
])(
  "accepts quantity %i",
  (quantity, expected) => {
    const result = calculateOrderTotal([
      { price: 1000, quantity }
    ]);

    expect(result).toBe(expected);
  }
);
```

Parameterized tests уменьшают повторение, когда одна логика проверяется на нескольких наборах данных.

19. Запустите watch mode

```
PowerShell
```

```
npm run test:watch
```

Измените `productService.js` и убедитесь, что Jest предлагает повторный запуск связанных tests.

20. Самостоятельное задание

10. Добавьте метод `delete(id)` в Product Service.
11. Если Product отсутствует, метод должен выбрасывать `Product not found`.
12. Если Product существует, Service должен вызвать `repository.delete(id)`.
13. Напишите positive unit test.
14. Напишите negative unit test.
15. Проверьте `toHaveBeenCalled()`.
16. Проверьте `not.toHaveBeenCalled()` для запрещённого сценария.
17. Добавьте `test.each()` минимум для двух boundary cases.
18. Запустите весь suite через `npm test`.
19. Убедитесь, что PostgreSQL не требуется для выполнения tests.

Требования к результату

- Jest установлен как dev dependency.
- Jest запускается в ESM-проекте.
- Создан `jest.config.js`.
- Tests используют `@jest/globals`.
- Используются `describe()` и `test()`.
- Используются `expect()` и несколько matchers.
- Есть async tests.
- Есть `beforeEach()`.
- Используется `jest.fn()`.
- Проверяются arguments и число вызовов mock.
- Product Service тестируется без PostgreSQL.
- Есть positive и negative Service tests.
- Все tests проходят через `npm test`.

Критерии оценивания

№	Критерий
1	Jest установлен и запускается
2	Настроен ESM запуск Jest
3	Используется <code>describe()</code>
4	Используется <code>expect()</code>
5	Применяется <code>toBe()</code>
6	Применяется <code>toEqual()</code>

№	Критерий
7	Проверяется toThrow()/rejects
8	Используется beforeEach()
9	Создан mock через jest.fn()
10	Используется mockResolvedValue()
11	Проверяется toHaveBeenCalledWith()
12	Проверяется toHaveBeenCalledTimes() или not.toHaveBeenCalled()
13	Product Service изолирован от Prisma/PostgreSQL
14	Есть positive и negative Service tests
15	Студент объясняет назначение dependency injection и mock functions